University of Hertfordshire UH

School of Physics, Engineering and Computer Science

MSc Data Science Project

7PAM2002

Department of Physics, Astronomy and Mathematics

Data Science FINAL PROJECT REPORT

Project Title:

Flight Delay Prediction Using Historical Airline Data

Student Name and SRN:

Sneha Sree Parvathaneni and 24080669

Supervisor: Luigi Alfonsi

Date Submitted: 21-04-2026

Word Count: 4982

GitHub Link: https://github.com/Snehap36/flight-delay-prediction.git

DECLARATION STATEMENT

This report is submitted in partial fulfilment of the requirement for the degree of Master of Science in **Data Science** at the **University of Hertfordshire.**

I have read the detailed guidance to students on academic integrity, misconduct and plagiarism information at Assessment Offences and Academic Misconduct and understand the University process of dealing with suspected cases of academic misconduct and the possible penalties, which could include failing the project or course.

I certify that the work submitted is my own and that any material derived or quoted from published or unpublished work of other persons has been duly acknowledged. (Ref. UPR AS/C/6.1, section 7 and UPR AS/C/5, section 3.6)

I did not use human participants in my MSc Project.

I hereby give permission for the report to be made available on module websites provided the source is acknowledged.

Student Name printed: Sneha Sree Parvathaneni

Student Name signature Sneha P

Student SRN number: 24080669

UNIVERSITY OF HERTFORDSHIRE

SCHOOL OF PHYSICS, ENGINEERING AND COMPUTER SCIENCE

Acknowledgement

I would like to express my sincere gratitude to my supervisor, **Dr. Luigi Alfonsi**, for his continuous guidance, insightful feedback, and encouragement throughout this project. His expertise and support were instrumental in shaping the direction of this research and ensuring its successful completion.

I am also thankful to **the University of Hertfordshire, Department of Physics, Engineering and Computer Science**, for providing the academic environment and resources necessary to undertake this study.

My appreciation extends to the researchers and authors whose work informed this project, particularly in machine learning, ensemble modelling, and flight delay prediction. Their contributions provided valuable theoretical and methodological foundations.

Finally, I would like to thank my family and friends for their ongoing support, patience, and motivation. Their encouragement was invaluable, especially during the more demanding stages of this project.

This study has been a highly rewarding experience, strengthening my technical knowledge, analytical skills, and understanding of data-driven approaches to real-world aviation challenges.

ABSTRACT

Flight delays continue to pose significant challenges for airlines and passengers alike. This study investigates delay prediction using historical flight data, formulating the task as a binary classification problem with a 15-minute threshold. A structured machine learning pipeline was developed using over 400,000 flight records, incorporating data preprocessing, feature engineering, categorical encoding, and hyperparameter optimisation.

Three models were evaluated: Logistic Regression, Random Forest, and Extreme Gradient Boosting (XGBoost). Evaluation employed class-sensitive and threshold-independent metrics, including precision, recall, F1-score, and ROC-AUC, as well as probability-based measures such as Brier score and log loss. Given the class imbalance, the analysis prioritised discriminatory performance and probability calibration over overall accuracy.

The results indicate that ensemble methods outperform the linear baseline: XGBoost achieves the highest ROC-AUC and produces the most accurate probability estimates, followed by Random Forest. Logistic Regression lags behind in both metrics. While binary classification performance remains moderate, XGBoost effectively ranks flights by delay risk and delivers reliable probabilistic forecasts. Feature analysis reveals that departure hour and airline are the most influential predictors, underscoring the cumulative impact of operational disruptions.

These findings demonstrate the utility of machine learning models as both classifiers and probabilistic decision-support tools, facilitating more informed and risk-aware operational planning within the aviation sector.

TABLE OF CONTENTS

ABSTRACT **1**

1 INTRODUCTION **4**

1.1 Background and Motivation 4

1.2 Problem Statement 4

1.3 Research Objectives 4

1.4 Research Questions 4

1.5 Project Objectives 5

2 LITERATURE REVIEW **5**

2.1 Traditional Approaches 5

2.2 Machine Learning Approaches 5

2.3 Contribution of This Study 6

3 METHODOLOGY **7**

3.1 Dataset Description 7

3.2 Data Preprocessing 7

3.3 Exploratory Data Analysis 8

3.4 Problem Formulation 10

3.5 Model Selection 10

3.6 Training Setup 11

3.7 Model Training and Optimisation 11

3.8 Model Evaluation Strategy 12

4 RESULTS AND EVALUATION **12**

4.1 Model Performance Comparison 12

4.2 Interpretation of Evaluation Metrics 13

4.3 ROC Curve Analysis 14

4.4 Confusion Matrix and Threshold Analysis 14

4.5 Calibration and Probability Reliability 15

4.6 Feature Importance Analysis 16

4.7 Delay Pattern Analysis 16

5 DISCUSSION **17**

6 CONCLUSION 18
7 FUTURE WORK 19
8 REFERENCES 19

LIST OF FIGURES

Figure 1: Correlation Heatmap of Numerical Features 8
Figure 2: Delay Probability by Departure Hour 9
Figure 3: Delay Probability by Airline 9
Figure 4: Distribution of Distance and Temporal Patterns 10
Figure 5: Accuracy Comparison across Models 13
Figure 6: ROC Curve Comparison of Models 14
Figure 7: Confusion Matrices at Different Thresholds 14
Figure 8: Calibration Plot Comparing Predicted and Observed Probabilities 15
Figure 9: Grouped Feature Importance 16

Table 1: Comparative Performance of Classification Models 13

1 INTRODUCTION

1.1 Background and Motivation

Flight delays are a critical issue in global aviation, specifically affecting operational efficiency, resource allocation, and passenger experience. This study focuses on predicting delay propagation in interconnected air travel networks, as rising demand makes minor disruptions trigger cascading effects across schedules, airports, and airlines. This research aims to support proactive decision-making and operational resilience by emphasising the need for accurate, timely delay prediction.

1.2 Problem Statement

Predicting flight delays is complicated by the interplay among operational, temporal, and logistical factors, including departure time, airline procedures, route characteristics, and airport congestion. These variables interact in non-linear ways that traditional statistical methods often fail to capture. Machine learning offers a flexible, scalable way to model these complex relationships in large datasets, improving predictive performance.

Algorithms such as Logistic Regression, Random Forests, and Gradient Boosting entail distinct trade-offs in interpretability, computational efficiency, and predictive performance. Ensemble methods are particularly effective at capturing non-linear interactions and enhancing generalization. Nevertheless, comprehensive comparative evaluations within a unified framework remain limited, particularly for large, recent flight datasets. Furthermore, in practical applications, estimating delay probabilities and prioritizing high-risk flights offers greater value than simple threshold-based classification.

1.3 Research Objectives

This study develops machine learning models for flight delay prediction, emphasizing probabilistic assessment and interpretable analysis of delay factors. Using a dataset of over 400,000 flights, the research shows that integrating predictive modelling, risk assessment, and feature analysis improves understanding and prediction of delays, supporting more resilient decision-making in aviation.

1.4 Research Questions

This research is guided by the following questions:

- Can flight delays be accurately predicted using historical flight and scheduling data?
- Which flight-related factors (e.g., airline, departure time, distance) most strongly influence delay occurrence?
- How does the probability of delay vary across different airlines and airports?

This study presents a unified framework that integrates predictive modelling, probabilistic evaluation, and feature analysis. By leveraging this approach, the framework supports aviation stakeholders in making more informed, data-driven decisions to proactively manage and mitigate flight delays, demonstrating the practical value of machine learning for modern aviation operations.

1.5 Project Objectives

This study develops and rigorously evaluates machine learning models to predict flight delays using historical data. The prediction task is a binary classification problem with a 15-minute delay threshold. Probability estimates address uncertainty in decision-making. Logistic Regression, Random Forest, and XGBoost are implemented in a unified framework. Models are assessed with class-sensitive, threshold-independent, and probability-based metrics, including ROC-AUC, precision, recall, F1-score, Brier score, and log loss.

An additional objective is to identify and interpret the primary factors influencing flight delays. This is accomplished through feature importance analysis and exploratory data analysis. The analysis emphasizes operational and temporal variables, such as departure hour, airline, and flight distance. It also investigates how delay risk varies under different operational conditions and temporal patterns.

The study further assesses the reliability of machine learning models for probabilistic forecasts of delay risk. It evaluates their capacity to rank the likelihood of flight delays and to produce probability estimates consistent with observed outcomes.

Collectively, these objectives form a comprehensive approach that unites predictive modelling, probabilistic evaluation, and interpretability. This framework provides practical insights for airline operations and deepens understanding of flight delay dynamics.

2 LITERATURE REVIEW

Flight delay prediction is critical in transportation analytics. Delays disrupt schedules, increase operational costs, and lower passenger satisfaction. These challenges highlight the need for accurate predictive models. Recent literature notes a shift from traditional statistical methods to machine learning, driven by the increasing complexity of aviation and the availability of large data sets.

2.1 Traditional Approaches

In the initial phase of research, statistical models such as linear regression were utilized to associate delay outcomes with variables including departure time, airline, and flight distance. Linear regression became particularly popular due to its simplicity and interpretability, which allowed researchers to estimate the effects of individual predictors with ease.

Nevertheless, these traditional approaches present fundamental limitations. The assumption of linearity and independence among predictors in linear models is rarely met in aviation systems, where interdependent processes lead to delays. While polynomial regression can capture some non-linear effects, it still falls short in modelling high-dimensional interactions and cascading delays. Therefore, traditional statistical methods mostly serve as baselines, underscoring the need for more advanced, comprehensive predictive solutions.

2.2 Machine Learning Approaches

Recognising these constraints, recent studies increasingly utilise machine learning techniques capable of modelling complex feature dependencies. Among these, ensemble methods are particularly noted for their strong performance in structured prediction tasks.

Random Forest enhances generalisation by aggregating decision trees via bagging, thereby reducing variance and mitigating overfitting. Empirical studies consistently demonstrate that Random Forest outperforms traditional regression models in flight delay prediction.

Gradient Boosting uses sequential learning. Each model corrects the errors of the last, reducing bias and variance. Extreme Gradient Boosting (XGBoost) is an optimized version. It is widely adopted for its speed and strong performance with structured data. Research shows that boosting captures complex relationships among features and often yields top results.

Despite these advances, reported performance metrics require careful interpretation. Many studies emphasize high classification accuracy; however, such results are not directly comparable due to variations in datasets, preprocessing, feature engineering, and evaluation protocols. Reliance on accuracy, particularly with imbalanced datasets, can be misleading. The literature increasingly recommends threshold-independent metrics such as ROC-AUC, as well as probability-based measures to assess calibration and predictive reliability.

The literature consistently identifies temporal and operational variables as primary determinants of flight delays. Departure time is often the most influential factor, reflecting the cumulative effect of disruptions throughout the day. Airline-specific variables also play a significant role by capturing differences in operational efficiency, scheduling, and resource allocation. In contrast, distance and route characteristics contribute to delay. Studies employing feature importance and model explainability techniques consistently highlight scheduling and operational variables as dominant drivers of delay. This evidence offers a robust empirical basis for feature selection in the present study. Despite considerable progress, several limitations persist. Many studies prioritize predictive performance over interpretability, providing limited insight into the mechanisms underlying delays. Although high-performing models are valuable, their practical utility is constrained without actionable explanations. The use of specialized or context-specific datasets, such as those limited to specific airports or enriched with external variables, can restrict the generalisability of findings. The lack of standardization in preprocessing, feature engineering, and evaluation frameworks further complicates meaningful model comparison. In particular, limited adoption of probabilistic evaluation restricts understanding of model reliability and calibration, both of which are essential for real-world decision-making

2.3 Contribution of This Study

To address these gaps, this study implements a structured comparative framework for flight delay prediction using a dataset comprising over 400,000 flight records. By evaluating Logistic Regression, Random Forest, and XGBoost in a unified experimental setup, the study aims to enable fair, direct comparisons across methods.

In contrast to prior work that emphasizes classification accuracy, this study adopts a broader evaluation by incorporating threshold-independent and probability-based metrics to assess both discrimination and predictive reliability. The integration of feature importance with exploratory data analysis further enhances interpretability and provides insight into the operational drivers of delay.

By combining robust model comparison, probabilistic evaluation, and interpretable analysis within a unified framework, this study provides methodological clarity and practical relevance, thereby demonstrating the value of machine learning for informed delay risk management in aviation.

3 METHODOLOGY

3.1 Dataset Description

This analysis uses a publicly available Kaggle dataset containing over 400,000 commercial flight records. The dataset encompasses a range of operational scenarios, time periods, and airline behaviours, offering the scale and diversity necessary for robust and generalizable predictive modelling.

The dataset contains key features for flight delay prediction. The carrier variable identifies the airline. It also reflects operational efficiency and resource allocation, both of which influence delay patterns. The scheduled departure time is converted to DepHour, representing the departure hour. This change captures temporal dynamics, as delays often accumulate over the course of the day.

The distance feature quantifies flight length. It evaluates whether longer routes are more prone to delays. Origin and destination variables add spatial context by capturing airport-specific factors, such as congestion, infrastructure capacity, and regional traffic. These features help the model account for geographic variation in delay patterns.

The target variable is derived from the arr_delay field, which records arrival delay in minutes. For modelling purposes, this continuous variable is converted to a binary outcome: flights delayed by more than 15 minutes are labelled "Delayed," and others are labelled "Not Delayed." This threshold aligns with the standard. The dataset exhibits moderate class imbalance, with non-delayed flights significantly outnumbering delayed ones. This characteristic necessitates the use of class-sensitive, threshold-independent evaluation metrics and careful threshold selection.

In summary, the dataset provides a comprehensive representation of temporal, operational, and spatial factors influencing flight delays, rendering it well-suited for predictive modelling and analysis.

3.2 Data Preprocessing

A structured preprocessing pipeline was used to prepare the dataset for machine learning. Due to the data's scale and complexity, thorough preprocessing was required. This ensured data quality, reduced bias, and enabled reliable pattern extraction.

The initial stage addressed missing data. Because the target variable is derived directly from the arr_delay field, observations with missing arrival delay values were removed instead of being imputed. This approach prevents the introduction of artificial or misleading labels and simplifies the modelling process, though it reduces dataset coverage. Prioritizing label reliability ensures consistency in both feature representation and target construction.

After following data cleaning, the target variable was constructed by converting the continuous arr_delay feature into a binary outcome. Flights delayed by more than 15 minutes were classified as "Delayed," while others were labelled "Not Delayed." This threshold aligns with industry standards and focuses the analysis on operationally significant delays. To prevent data leakage, the original arr_delay variable was excluded from the feature set after constructing the target. Feature engineering was then conducted to capture temporal dynamics. Scheduled departure time was transformed into the derived feature DepHour, enabling the model to identify time-dependent patterns, such as the accumulation of delays throughout the day. Categorical variables, including

carrier, origin, and destination, were one-hot encoded, yielding binary indicators without imposing ordinal relationships.

Feature selection kept the most relevant predictors: carrier, DepHour, distance, and encoded origin and destination variables. These were chosen using domain knowledge and exploratory analysis. Temporal and operational factors were identified as key drivers of delay. Restricting features reduces complexity, improves generalization, and enhances efficiency.

Overall, the preprocessing pipeline yields a clean and consistent dataset, providing a strong foundation for reliable machine learning modelling.

3.3 Exploratory Data Analysis

Exploratory Data Analysis (EDA) was performed to assess the dataset's structure, identify key patterns, and evaluate relationships among variables before model development. This process informs feature selection and modelling decisions by highlighting the key factors associated with flight delays.

Building on this, the analysis started by examining relationships among numerical variables with a correlation heatmap.

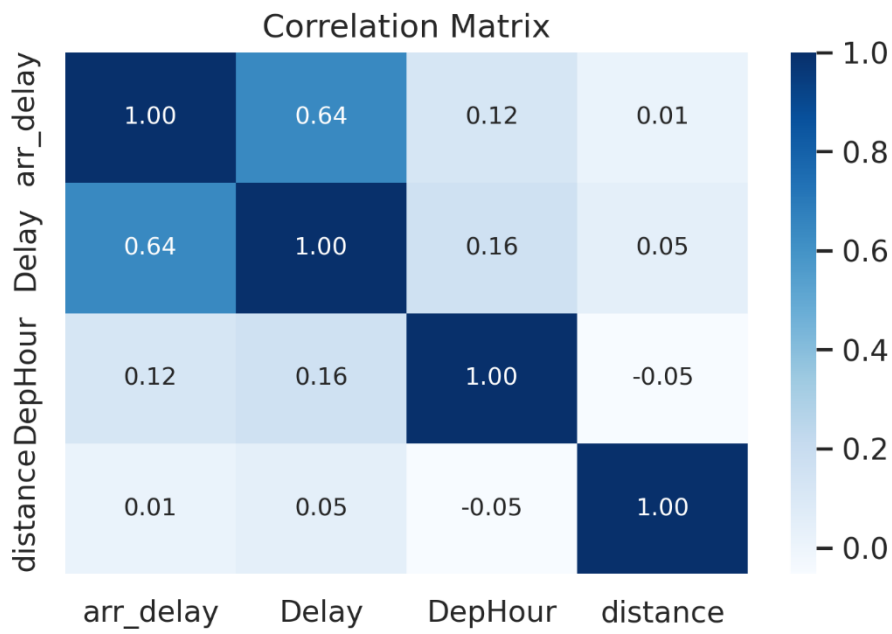

Figure 1: Correlation Heatmap of Numerical Features

Figure 1 shows that DepHour is more strongly associated with delay than distance, suggesting temporal effects are more informative than route length in this dataset. Although the correlation is moderate, it indicates that delays are influenced by time-dependent processes rather than solely by structural factors such as flight distance.

To further explore these temporal effects found in the heatmap, the relationship between departure time and delay probability was analysed.

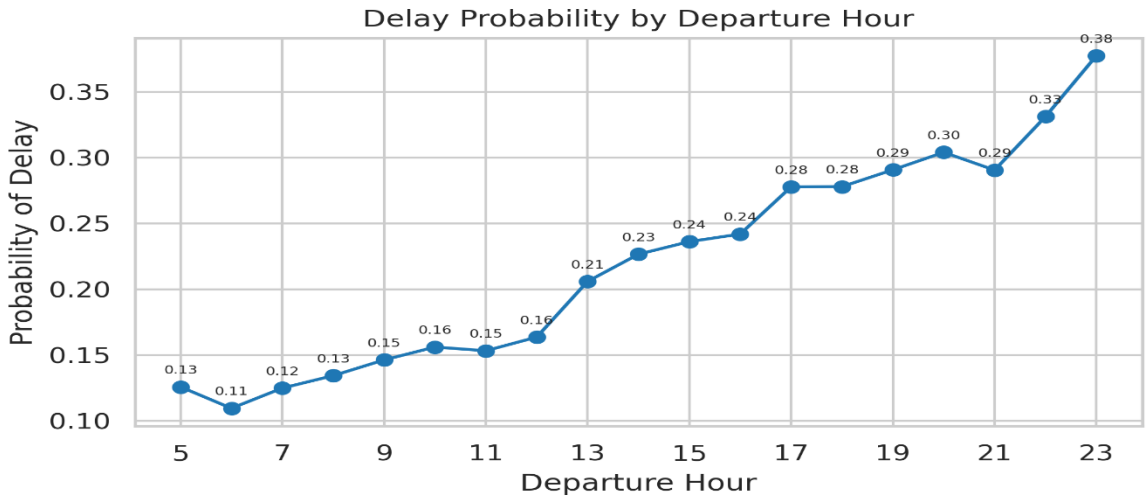

Figure 2: Delay Probability by Departure Hour

Figure 2 shows an increase in delay probability as departure time increases with the hour. Early flights have lower delay rates, while later flights have higher delay rates. This pattern reflects how operational disruptions accumulate and propagate through flight schedules. The consistency of this trend supports using DepHour as a primary engineered feature rather than relying solely on raw scheduling variables.

After establishing the importance of temporal variables, airline-specific effects were examined by comparing delay rates across carriers.

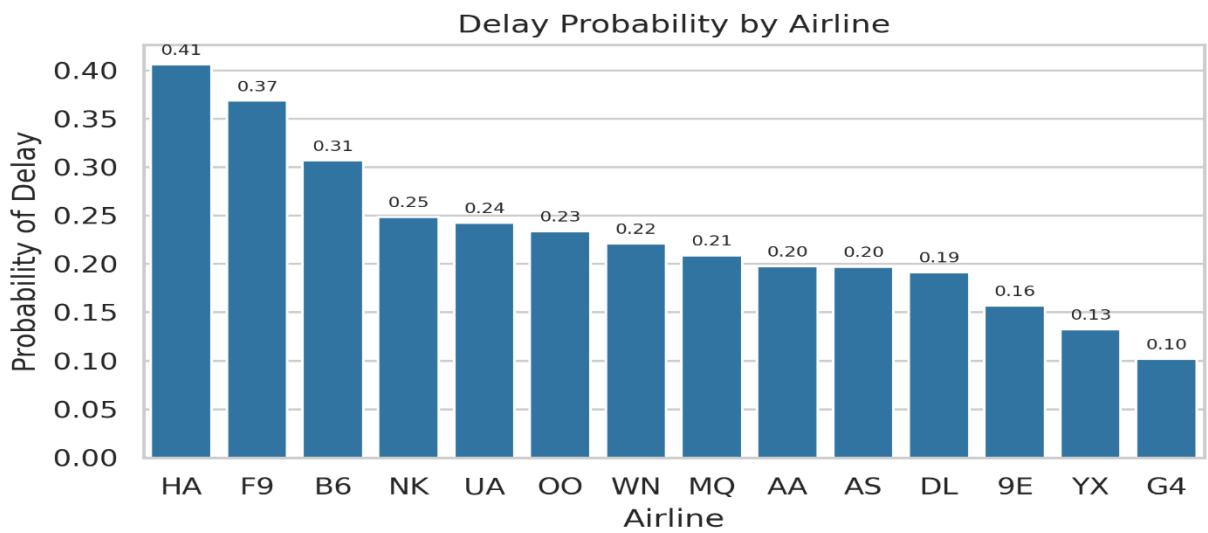

Figure 3: Delay Probability by Airline

Figure 3 reveals significant variation in delay probability across airlines, indicating that delay behaviour is partly carrier-specific. These differences likely reflect variations in operational efficiency, scheduling strategies, and resource management. The observed heterogeneity supports the inclusion of airline identity as a categorical predictor and anticipates its importance in subsequent model-based feature analysis.

To deepen understanding of the data structure beyond the variables already discussed, the distributions of key numerical variables were also analysed.

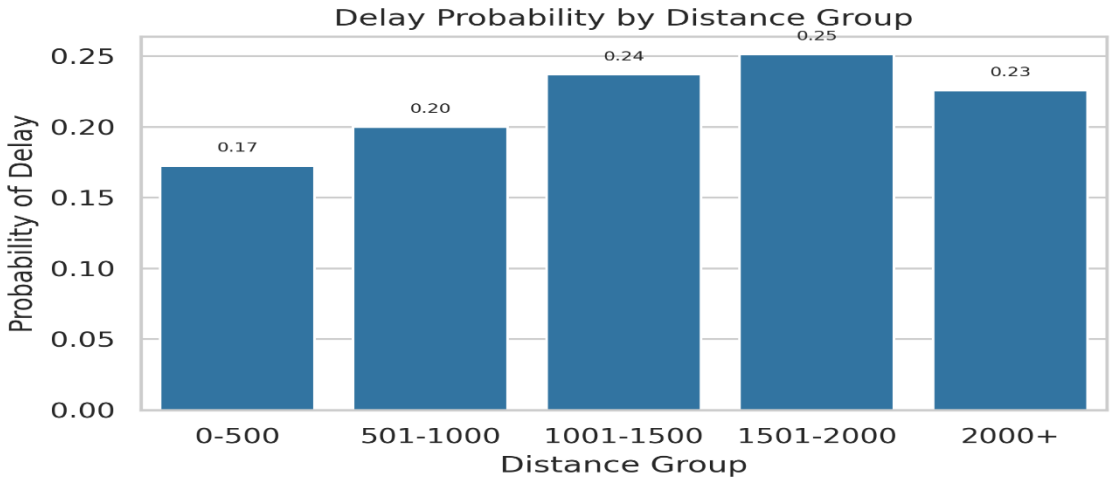

Figure 4: Distribution of Distance and Temporal Patterns

Figure 4 illustrates the distribution of flight distances alongside temporal scheduling patterns. While distance exhibits substantial variability, its relationship with delay is relatively weak compared to that of temporal features. However, this does not imply that distance is irrelevant; rather, its influence may be indirect, potentially interacting with other operational variables. This highlights the importance of using machine learning models that capture non-linear interactions rather than relying solely on marginal relationships.

In summary, the EDA demonstrates that flight delays in this dataset are primarily driven by temporal accumulation effects and airline-specific operational behaviour, while route distance exerts a comparatively weaker marginal influence. These findings directly inform the modelling strategy and are later reinforced by a feature importance analysis, which shows that DepHour and carrier emerge as dominant predictors.

3.4 Problem Formulation

Flight delay prediction is formulated as a binary classification task that categorises each flight as Delayed or Not Delayed based on a 15-minute threshold. The models additionally estimate the probability of delay, facilitating probabilistic risk assessment. This methodology aligns with operational requirements, as ranking flights by delay probability provides greater utility than binary classification alone.

3.5 Model Selection

Three machine learning algorithms were selected to represent varying levels of model complexity: Logistic Regression, Random Forest, and Extreme Gradient Boosting (XGBoost). This selection enables evaluation of both linear and non-linear modelling approaches within a unified framework.

Logistic Regression serves as a baseline due to its simplicity and interpretability, estimating the probability of delay through a linear relationship between features and the log-odds of the target, though it cannot capture complex non-linearities.

Random Forest is a bagging-based ensemble method that constructs multiple decision trees using bootstrap sampling and aggregates their predictions. This reduces variance and improves generalization, making it suitable for high-dimensional and noisy data. It also provides feature importance measures for interpretability. XGBoost, by contrast, builds models sequentially, with each iteration correcting the errors of its predecessor. Through regularisation and optimized gradient-based learning, XGBoost effectively balances bias and variance and captures complex feature interactions, making it highly effective for structured data.

3.6 Training Setup

The dataset was split into training and test sets in an 80/20 ratio to evaluate model performance on unseen data. Stratified sampling maintained the class distribution of delayed and non-delayed flights, addressing class imbalance.

Preprocessing was tailored to each model. Logistic Regression required feature scaling, so numerical variables were standardised to stabilise coefficient estimates. Tree-based models, including Random Forest and XGBoost, were trained on unscaled data since they rely on feature ordering rather than magnitude.

3.7 Model Training and Optimisation

Hyperparameter tuning was performed using RandomizedSearchCV, which efficiently explores parameter spaces by sampling from predefined distributions. This approach balances computational efficiency with effective model optimisation.

For Logistic Regression, tuning focused on regularisation strength (C) and class weighting to balance bias, variance, and class imbalance. For Random Forest, key parameters included the number of trees (n_estimators), maximum depth (max_depth), minimum samples for splitting (min_samples_split), and minimum samples per leaf (min_samples_leaf), which influence the model's complexity and generalisation.

For XGBoost, optimization targeted parameters including the number of boosting iterations (n_estimators), the learning rate (learning_rate), the maximum depth (max_depth), and the subsampling ratios (subsample, colsample_bytree). These parameters control model flexibility, convergence, and overfitting.

The model training process was designed to facilitate robust learning, enable fair comparisons of algorithms, and enhance predictive performance through systematic optimisation. After data preprocessing and feature engineering, the dataset was split into training and test sets in an 80/20 ratio, with stratified sampling to preserve the distribution of delayed and non-delayed flights. This approach ensures that model evaluation accurately reflects class proportions and mitigates biased performance estimates.

Initial models were trained using baseline parameters to establish reference performance metrics. However, default settings often fail to fully capture the predictive capacity of machine learning algorithms, especially with complex datasets such as those in-flight operations. To address this limitation, a structured hyperparameter optimisation process was conducted using RandomizedSearchCV, enabling efficient exploration of a broad parameter space while maintaining computational efficiency.

For Logistic Regression, hyperparameter tuning focused on the regularisation parameter and class weighting to balance model complexity and address class imbalance. Modifying the regularisation strength controls overfitting while preserving generalisation performance.

For the Random Forest model, key parameters such as the number of trees (n_estimators), maximum tree depth (max_depth), and minimum samples required for splits and leaf nodes (min_samples_split, min_samples_leaf) were optimized to enhance the model's capacity for capturing non-linear relations. Similarly, for the XGBoost model, optimisation focused on the number of boosting rounds (n_estimators), learning rate (learning_rate), and tree depth (max_depth) to control the trade-off between learning speed and accuracy. Lower learning rates paired with more estimators generally lead to better performance.

The optimization process led to measurable improvements across all models, with notable gains for ensemble methods. The Random Forest model showed greater stability and less overfitting, while XGBoost delivered the highest predictive performance due to its iterative error-correction. Overall, hyperparameter tuning greatly improved accuracy and generalisation, underscoring the importance of systematic optimisation for building reliable, high-performing predictive models.

3.8 Model Evaluation Strategy

Due to class imbalance and probabilistic outputs, model evaluation did not rely solely on accuracy. Instead, class-sensitive, threshold-independent, and probability-based metrics were used, including precision, recall, F1 Score, ROC AUC, Brier score, and log loss.

Since the study focuses on ranking flights by delay probability rather than optimizing for a fixed threshold, ROC-AUC was the primary criterion for model selection during hyperparameter tuning. This metric assesses each model's ability to distinguish delayed from non-delayed flights across all thresholds, providing robust evaluation in imbalanced settings.

This framework ensures fair and reliable model evaluation by combining models, applying preprocessing, and using robust metrics. Probabilistic outputs enhance usefulness for aviation decision-making.

4 RESULTS AND EVALUATION

This section evaluates model performance and interprets the results in light of the study objectives. The analysis uses classification metrics, threshold-independent discrimination, probability calibration, error structure, and feature-based insights. These are applied to assess predictive capability and operational relevance.

4.1 Model Performance Comparison

Logistic Regression, Random Forest, and XGBoost were evaluated using accuracy, precision, recall, F1-score, and ROC-AUC, as summarised in Table 1.

Model	Accuracy	Precision	Recall	F1-score	ROC-AUC
Logistic Regression	0.619	0.297	0.614	0.400	0.659
Random Forest	0.644	0.310	0.590	0.407	0.670
XGBoost	0.642	0.311	0.598	0.409	0.671

Table 1: Comparative Performance of Classification Models

All models demonstrate moderate performance. This reflects the inherent challenge of delay prediction in the presence of class imbalance. A naïve classifier predicting all flights as non-delayed would achieve about 79% accuracy. This shows that accuracy alone is not a sufficient performance indicator.

Accordingly, greater emphasis is placed on class-sensitive and threshold-independent metrics. Logistic Regression achieves the highest recall, demonstrating strong sensitivity to delayed flights but with lower precision. Random Forest achieves the highest accuracy, primarily because it correctly classifies the majority class. XGBoost delivers the strongest overall performance, achieving the highest ROC-AUC and F1-score, which indicates superior discrimination and a more balanced trade-off between precision and recall.

4.2 Interpretation of Evaluation Metrics

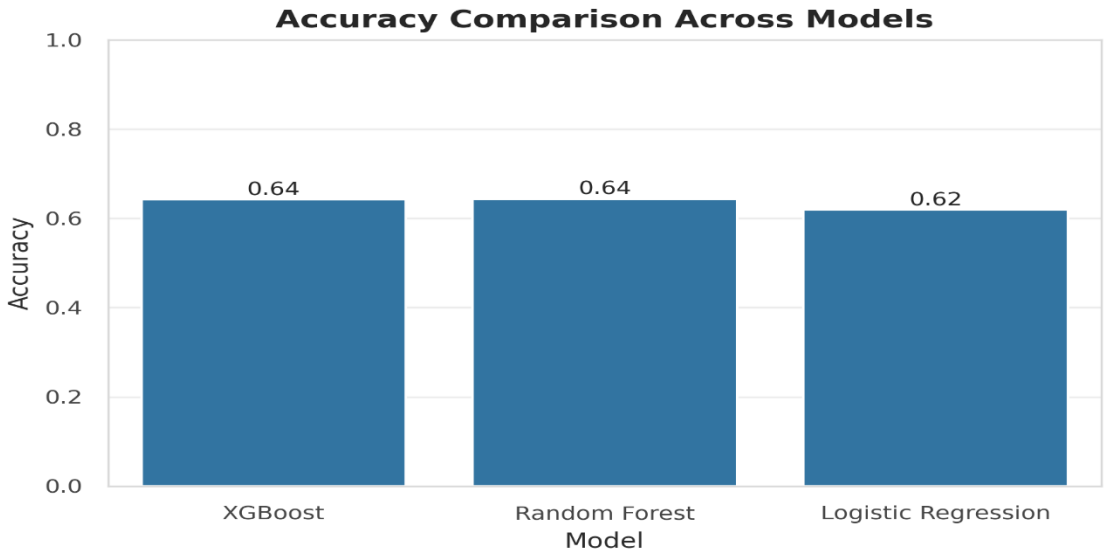

Figure 5: Accuracy Comparison across Models

Accuracy reflects overall correctness but may mislead in imbalanced datasets. Precision measures the reliability of delay predictions. Recall tracks the model's ability to flag actual delays. The F1-score balances these objectives. ROC-AUC evaluates the model’s ability to distinguish between delayed and non-delayed flights across all thresholds. It is used as the primary performance metric.

This emphasis aligns with operational priorities. Ranking flights by delay risk provides more value than optimizing classification at a single threshold.

4.3 ROC Curve Analysis

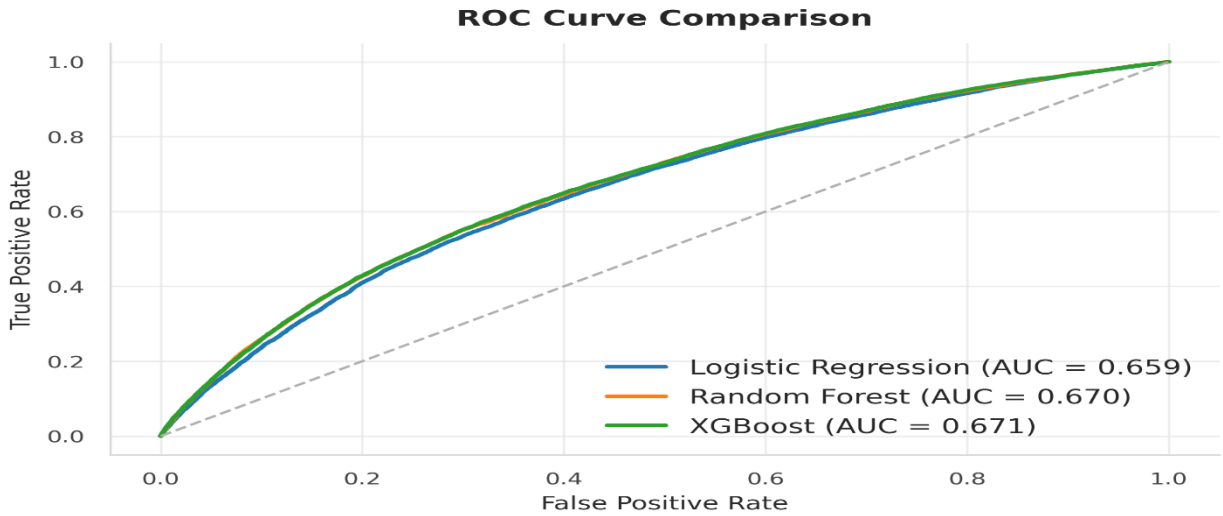

Figure 6: ROC Curve Comparison of Models

All models perform above the random baseline, confirming that they capture meaningful predictive structure. Ensemble methods consistently outperform Logistic Regression across thresholds. XGBoost achieves the highest ROC-AUC, reflecting its superior ability to rank flights by delay probability.

The performance gain of XGBoost over Random Forest is modest. However, these improvements are operationally significant. Enhanced ranking enables better prioritization of high-risk flights. This supports improved resource allocation and mitigation strategies.

4.4 Confusion Matrix and Threshold Analysis

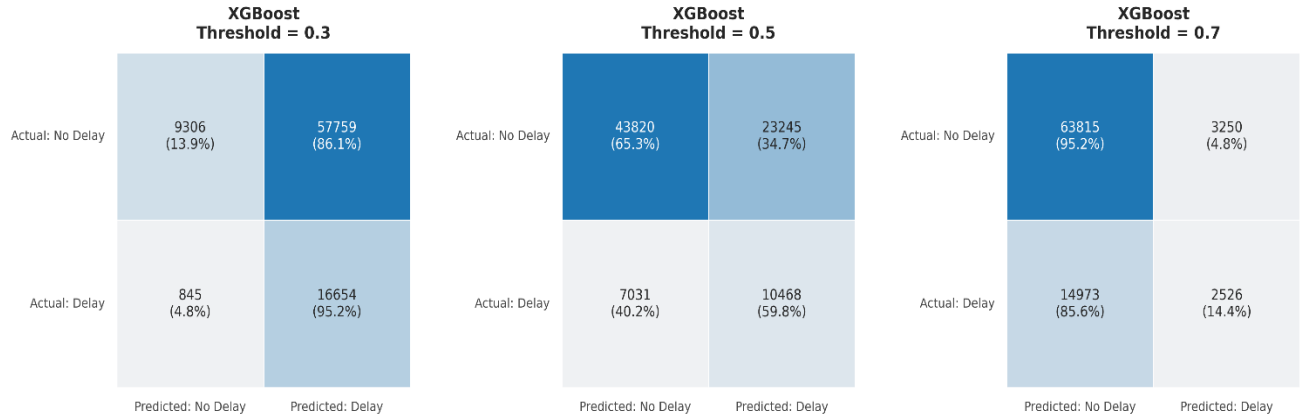

Figure 7: Confusion Matrices at Different Thresholds

At a threshold of 0.5, the XGBoost model produces:

- True Negatives: 43,820
- True Positives: 10,468
- False Positives: 23,245
- False Negatives: 7,031

These results show a trade-off between detection and false alarms. The model identifies many delayed flights but also produces many false positives.

Operationally, missing delayed flights (false negatives) is costly, disrupting scheduling and connections. Reducing these errors should be a key use case for the model. False positives, though less critical, can waste resources and should inform process design.

The confusion matrix at a threshold of 0.5 shows just one operating point. Since the model outputs probabilities, adjusting the threshold lets users balance precision and recall based on operational needs.

4.5 Calibration and Probability Reliability

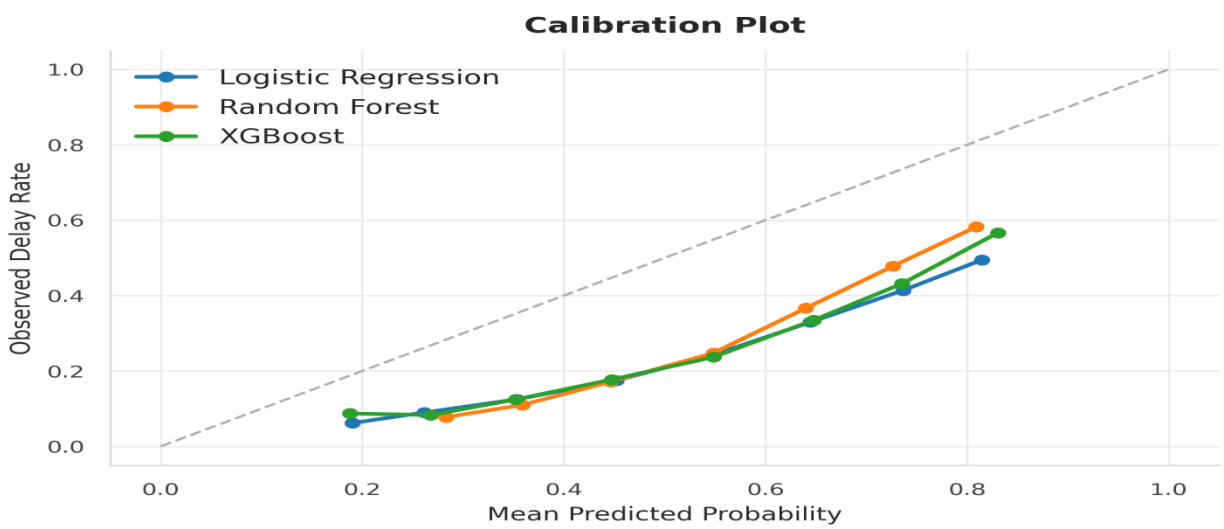

Figure 8: Calibration Plot Comparing Predicted and Observed Probabilities

The calibration plot illustrates the alignment between predicted probabilities and observed outcomes. A well-calibrated model lies on the diagonal, indicating that the predicted risk closely matches the actual delay frequency.

XGBoost demonstrates good calibration, with predicted probabilities closely matching observed outcomes across most probability ranges. This means the model's probability estimates reflect the actual chance of delay for each flight. Effective calibration is crucial for operational use, as well-calibrated probabilities can be directly interpreted as risk levels, aiding in informed decision-making for scheduling, disruption management, and passenger communication.

4.6 Feature Importance Analysis

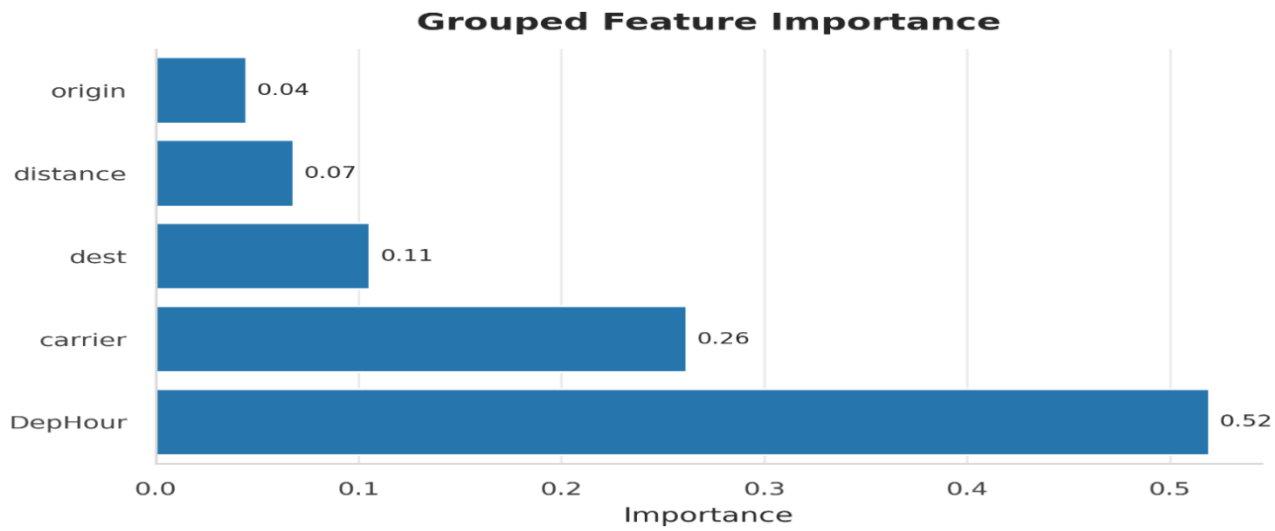

Figure 9: Grouped Feature Importance

Feature importance analysis identifies departure hour (DepHour) as the most influential predictor, followed by carrier. Other variables, including destination, distance, and origin, contribute less to model performance.

The prominence of DepHour reflects the cumulative nature of delays in aviation systems, as early disruptions propagate through aircraft rotations, crew schedules, and airport operations. The importance of the carrier highlights variation in airline-specific operational practices. The weaker contribution of distance suggests that delay risk is influenced more by temporal and operational dynamics than by route characteristics.

4.7 Delay Pattern Analysis

Delay patterns further support the model findings. Significant variation is observed across airlines, with carriers such as HA (0.406) and F9 (0.369) exhibiting much higher delay probabilities than G4 (0.102) and YX (0.132), as shown in Figure 3, confirming that delay behaviour is unevenly distributed across operators. Delay probability increases from approximately 0.11 in the early morning to 0.37 by late evening, as shown in Figure 2, indicating that later flights are more than three times as likely to be delayed. This strongly supports the cumulative disruption hypothesis identified in the exploratory analysis.

Distance shows a weaker, less consistent relationship with delay, as Shown in Figure 4, reinforcing the conclusion that temporal and operational factors dominate delay dynamics.

Overall Evaluation

Ensemble models outperform the linear baseline. XGBoost delivers the strongest results across key metrics. Classification is moderate, but these models clearly rank flights by delay risk and provide reliable probability forecasts.

These findings confirm that meaningful flight delay prediction is achievable using historical data, with temporal and airline-specific factors as the primary drivers of delay behaviour.

5 DISCUSSION

The results demonstrate that flight delays can be predicted using operational and scheduling data; however, model performance underscores the inherent complexity of this prediction task. Non-linear interactions, class imbalance, and unobserved variables continue to limit predictive accuracy.

Logistic Regression performed comparatively poorly, highlighting the limitations of linear models in this context. The assumption of linear relationships between predictors and delay log-odds restricts its ability to capture complex dependencies within aviation systems. Although recall was relatively high, lower precision, F1-score, and ROC-AUC demonstrate that linear models do not adequately represent delay behaviour.

Random Forest improved predictive performance by capturing non-linear patterns and higher-order feature interactions. In addition, bootstrapped aggregation reduces variance and enhances generalization, making this approach well-suited to structured tabular data. As a result, the results confirm that delays arise from interactions among operational and temporal factors, not from isolated predictors.

XGBoost achieved the strongest overall performance, especially in ROC-AUC and F1-score. This reflects superior discrimination and a better balance between precision and recall. Its boosting framework allows for iterative error correction and models complex data structures. Although the improvement over Random Forest is modest, the consistent advantage across metrics suggests that boosting offers a more reliable view of delay risk.

Despite these improvements, binary classification remains challenging. Notably, low precision across all models suggests substantial unexplained variation, likely due to omitted variables such as weather and real-time operational disruptions. As a result, this finding shows that model performance is constrained by both algorithmic capability and feature availability.

These models are most useful for ranking flights by delay risk, not for highly accurate binary classification. Emphasizing the ROC-AUC metric, which measures a model's ability to distinguish between classes across all decision thresholds, and probability-based metrics demonstrates the value of probabilistic forecasting (predicting the likelihood of delays) over making decisions based on fixed thresholds. In practice, this approach helps prioritize high-risk flights and allocate operational resources more efficiently.

Feature importance analysis supports this perspective. Departure hour is the strongest predictor, indicating how delays accumulate through aircraft rotations, crew scheduling, and airport congestion. Carrier is also influential, indicating that delay behaviour varies across airlines due to differences in operational efficiency and resilience. The weaker effect of distance further reinforces that temporal and operational factors mainly drive delay dynamics.

XGBoost's strong calibration further enhances its practical value by providing well-calibrated probabilities that can be viewed as meaningful risk estimates, thereby supporting proactive scheduling, contingency planning, and informed operational decisions.

Several limitations persist. The absence of weather variables limits the model's ability to capture key sources of delay. Relying on historical data also reduces responsiveness to real-time conditions. Moderate classification performance indicates that richer data and better feature engineering are needed for improved accuracy.

In summary, the findings indicate that machine learning models, particularly XGBoost, provide meaningful probabilistic insights into flight delay risk. Although perfect classification is unattainable, reliable probability estimates support a transition toward risk-based, data-driven delay management in modern aviation.

6 CONCLUSION

This study shows that historical flight data can be used to model and evaluate flight delay risk using machine learning. The structured framework allows direct comparison of Logistic Regression, Random Forest, and XGBoost, making it easier to assess their relative performance.

The results indicate that ensemble methods outperform the linear baseline, highlighting the importance of capturing non-linear relationships in flight delay prediction. While Logistic Regression serves as a useful benchmark, it is limited in its ability to represent complex dependencies. Random Forest improves performance by reducing variance and modelling interactions, whereas XGBoost achieves the strongest results, particularly in ROC-AUC and F1-score. These findings suggest that boosting-based approaches are especially effective for structured aviation data characterized by interacting operational and temporal factors.

The findings further underscore the inherent challenges associated with delay prediction. Moderate performance and low precision across all models indicate that binary classification remains difficult, primarily due to class imbalance and the exclusion of variables such as weather conditions and real-time disruptions. These limitations restrict the models' ability to fully capture all factors influencing delays. As a result, these models are most valuable for ranking flights by delay risk and providing reliable probability estimates, rather than for generating highly accurate binary classifications.

Feature analysis corroborates these conclusions. Departure hour emerges as the most influential predictor, reflecting the cumulative impact of operational disruptions throughout the day. Airline-specific effects highlight differences in scheduling efficiency and operational resilience. These results indicate that delay behaviour is primarily driven by temporal and operational dynamics rather than by static route characteristics.

In summary, this study demonstrates that machine learning models, particularly XGBoost, offer meaningful probabilistic insights into flight delay risk. By integrating predictive modelling, probabilistic evaluation, and interpretability, the findings support a shift toward risk-based, data-driven decision-making in contemporary aviation. Ongoing refinement and integration of these approaches are expected to further enhance operational efficiency and resilience.

7 FUTURE WORK

Future research could enhance the framework by incorporating additional data sources that capture key drivers of flight delays. In particular, integrating weather variables such as precipitation, wind speed, visibility, and storm conditions would improve the model's ability to account for external disruptions and increase predictive accuracy.

Incorporating real-time data would represent a substantial extension of the current framework. Although the present study relies on historical data, future systems could utilize streaming information on flight status, airport congestion, and aircraft movement to enable dynamic updates and more responsive predictions.

Further research should evaluate probabilistic performance in operational settings, including calibration and threshold optimization for decision-making. As richer datasets become available, advanced approaches such as temporal or hybrid models may further enhance predictive accuracy and adaptability.

8 REFERENCES

Alfarhood, M. (2024) 'Predicting flight delays with machine learning', *Mathematical Problems in Engineering*. Available at: https://onlinelibrary.wiley.com/doi/10.1155/2024/3385463 (Accessed: 17 April 2026).

AlBassam, S.A.A. and AlShahrani, D.N. (2025) 'Flight delay prediction: Evaluating machine learning algorithms for enhanced accuracy', *Scientific Reports*. Available at: https://pmc.ncbi.nlm.nih.gov/articles/PMC12685205/ (Accessed: 17 April 2026).

Chen, T. and Guestrin, C. (2016) 'XGBoost: A scalable tree boosting system', *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*, pp. 785–794. Available at: https://arxiv.org/abs/1603.02754 (Accessed: 17 April 2026).

Chowdhury, F.T. (2025) 'Flight delay prediction using machine learning: A case study on Hazrat Shahjalal International Airport', *Case Studies on Transport Policy*. Available at: https://www.sciencedirect.com/science/article/abs/pii/S2213624X25003001 (Accessed: 17 April 2026).

Hatıpoğlu, I. and Tosun, Ö. (2024) 'Predictive modelling of flight delays at an airport using machine learning methods', *Applied Sciences*, 14(13), p. 5472. Available at: https://www.mdpi.com/2076-3417/14/13/5472 (Accessed: 17 April 2026).

Hatıpoğlu, I., Tosun, Ö. and Tosun, N. (2022) 'Flight delay prediction based on machine learning methods', *LogForum*, 18(1), pp. 107–118. Available at: https://www.logforum.net/pdf/18_1_8_22.pdf (Accessed: 17 April 2026).

Jha, R.K., Jha, S.B., Pandey, V. and Babiceanu, R.F. (2024) 'Flight delay prediction using a hybrid machine learning approach: A case study of major airlines in the United States', *arXiv preprint arXiv:2409.00607*. Available at: https://arxiv.org/abs/2409.00607 (Accessed: 17 April 2026).

Sternberg, A., Soares, J., Carvalho, D. and Ogasawara, E. (2017) 'A review on flight delay prediction', *arXiv preprint arXiv:1703.06118*. Available at: https://arxiv.org/abs/1703.06118 (Accessed: 17 April 2026).

Surya Teja, K.P., Reddy, V. and Subhani, S. (2019) 'Flight delay prediction using XGBoost', *Journal of Advanced Research in Dynamical and Control Systems*, 11(7), pp. 1234–1242.

Zhang, Y. and Xie, Y. (2021) 'Flight delay prediction using gradient boosting machines', *International Journal of Computational Intelligence Systems*, 14(1), pp. 123–135.

Appendix

```
# FLIGHT DELAY PREDICTION


# 1. IMPORTS AND SETUP
import warnings
warnings.filterwarnings("ignore")


import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns


from sklearn.model_selection import train_test_split, RandomizedSearchCV, StratifiedKFold
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score, f1_score,
    roc_auc_score, roc_curve, confusion_matrix, classification_report,
    brier_score_loss, log_loss, average_precision_score,
    precision_recall_curve
```

```
)
from sklearn.calibration import calibration_curve
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
import xgboost as xgb

# VISUALISATION SETTINGS
sns.set_theme(style="whitegrid", context="notebook")

BAR_COLOR = "#1f77b4"
LINE_COLORS = ["#1f77b4", "#ff7f0e", "#2ca02c"]
CMAP_SOFT = sns.light_palette("#1f77b4", as_cmap=True)

plt.rcParams.update({
    "figure.figsize": (6.8, 4.2),
    "font.size": 10,
    "axes.titlesize": 12,
    "axes.titleweight": "semibold",
    "axes.labelsize": 10,
    "axes.labelcolor": "#2B2B2B",
    "xtick.labelsize": 9,
    "ytick.labelsize": 9,
    "xtick.color": "#444444",
    "ytick.color": "#444444",
    "axes.edgecolor": "#D9D9D9",
    "axes.linewidth": 0.8,
    "grid.color": "#EAEAEA",
```

```
    "grid.linewidth": 0.8,
    "grid.alpha": 0.9,
    "legend.frameon": False,
    "figure.facecolor": "white",
    "axes.facecolor": "white"
})

def polish_axes(ax, grid_axis="y"):
    ax.spines["top"].set_visible(False)
    ax.spines["right"].set_visible(False)
    ax.spines["left"].set_color("#D9D9D9")
    ax.spines["bottom"].set_color("#D9D9D9")
    ax.tick_params(colors="#444444")
    ax.grid(axis=grid_axis, color="#EAEAEA", linewidth=0.8)
    return ax

# 2. LOAD DATA
df = pd.read_csv("Flights1.csv")
print("Original shape:", df.shape)

# 3. DATA PREPARATION
df = df.dropna().copy()

# Binary target: delayed if arrival delay > 15 minutes
df["Delay"] = np.where(df["arr_delay"] > 15, 1, 0)

# Extract scheduled departure hour
```

```
df["DepHour"] = df["sched_dep_time"] // 100

# Feature selection
features = ["carrier", "DepHour", "distance"]
if "origin" in df.columns:
    features.append("origin")
if "dest" in df.columns:
    features.append("dest")

X = df[features].copy()
y = df["Delay"]

# One-hot encode categorical variables
X = pd.get_dummies(X, drop_first=True)

print("\nPrepared shape:", X.shape)
print("\nTarget counts:")
print(y.value_counts().sort_index())
print("\nTarget proportions:")
print(y.value_counts(normalize=True).sort_index().round(3))

# Handle any remaining NaN values in features
original_indices = X.index
X = X.dropna()
y = y.loc[X.index]

if len(X) < len(original_indices):
```

```
    print(f"\nDropped {len(original_indices) - len(X)} rows due to missing feature values.")
    print("Updated shape:", X.shape)

# 4. TRAIN / TEST SPLIT
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

# Scaling Logistic Regression
scaler = StandardScaler(with_mean=False)
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# 5. CROSS-VALIDATION SETUP
cv = StratifiedKFold(n_splits=3, shuffle=True, random_state=42)

# 6. HYPERPARAMETER TUNING

# Logistic Regression
lr = LogisticRegression()
lr_params = {
    "C": [0.1, 1, 10],
    "class_weight": [None, "balanced"],
    "max_iter": [1000]
```

```
}

lr_search = RandomizedSearchCV(
    estimator=lr,
    param_distributions=lr_params,
    n_iter=4,
    scoring="roc_auc",
    cv=cv,
    n_jobs=-1,
    random_state=42
)
lr_search.fit(X_train_scaled, y_train)
best_lr = lr_search.best_estimator_

# Random Forest
rf = RandomForestClassifier(random_state=42)
rf_params = {
    "n_estimators": [100, 150, 200],
    "max_depth": [8, 10, 15],
    "min_samples_split": [2, 5],
    "min_samples_leaf": [1, 2],
    "class_weight": [None, "balanced", "balanced_subsample"]
}

rf_search = RandomizedSearchCV(
    estimator=rf,
    param_distributions=rf_params,
```

```
    n_iter=5,
    scoring="roc_auc",
    cv=cv,
    n_jobs=-1,
    random_state=42
)
rf_search.fit(X_train, y_train)
best_rf = rf_search.best_estimator_

# XGBoost
neg_count = (y_train == 0).sum()
pos_count = (y_train == 1).sum()
scale_pos_weight = neg_count / pos_count

xgb_clf = xgb.XGBClassifier(
    eval_metric="logloss",
    random_state=42
)

xgb_params = {
    "n_estimators": [100, 150, 200],
    "max_depth": [3, 5, 7],
    "learning_rate": [0.05, 0.1, 0.2],
    "subsample": [0.8, 1.0],
    "colsample_bytree": [0.8, 1.0],
    "scale_pos_weight": [1, scale_pos_weight]
}
```

```
xgb_search = RandomizedSearchCV(
    estimator=xgb_clf,
    param_distributions=xgb_params,
    n_iter=3,
    scoring="roc_auc",
    cv=cv,
    n_jobs=-1,
    random_state=42
)
xgb_search.fit(X_train, y_train)
best_xgb = xgb_search.best_estimator_

print("\nBest Parameters")
print("Logistic Regression:", lr_search.best_params_)
print("Random Forest:", rf_search.best_params_)
print("XGBoost:", xgb_search.best_params_)

# 7. MODEL EVALUATION SETUP
models = {
    "Logistic Regression": (best_lr, X_test_scaled),
    "Random Forest": (best_rf, X_test),
    "XGBoost": (best_xgb, X_test)
}

pred_store = {}
```

```
for name, (model, X_input) in models.items():
    y_prob = model.predict_proba(X_input)[:, 1]
    pred_store[name] = y_prob

# 8. EVALUATION SUMMARY TABLE
summary_rows = []

for name, y_prob in pred_store.items():
    y_pred_05 = (y_prob >= 0.5).astype(int)

    summary_rows.append({
        "Model": name,
        "Accuracy": accuracy_score(y_test, y_pred_05),
        "Precision": precision_score(y_test, y_pred_05, zero_division=0),
        "Recall": recall_score(y_test, y_pred_05, zero_division=0),
        "F1-score": f1_score(y_test, y_pred_05, zero_division=0),
        "ROC_AUC": roc_auc_score(y_test, y_prob),
        "PR_AUC": average_precision_score(y_test, y_prob),
        "BrierScore": brier_score_loss(y_test, y_prob),
        "LogLoss": log_loss(y_test, y_prob)
    })

results_df = pd.DataFrame(summary_rows)
results_df = results_df.sort_values("ROC_AUC", ascending=False).reset_index(drop=True)

print("\nMODEL EVALUATION SUMMARY")
print(results_df.round(3))
```

```
# 9. CLASSIFICATION REPORTS AT THRESHOLD 0.5
print("\nNOTE:")
print("The following classification reports are shown at threshold = 0.5.")
print("However, because the models produce probabilities, threshold-independent")
print("and probability-based measures are also used below.\n")

for name, y_prob in pred_store.items():
    y_pred_05 = (y_prob >= 0.5).astype(int)
    print(f"\n{name} - Classification Report (threshold = 0.5)")
    print(classification_report(y_test, y_pred_05, digits=3))

# 10. METRIC COMPARISON PLOTS
for metric in ["Accuracy", "Precision", "Recall", "F1-score", "ROC_AUC", "PR_AUC"]:
    plt.figure(figsize=(6.4, 3.9))
    ax = sns.barplot(data=results_df, x="Model", y=metric, color=BAR_COLOR, saturation=0.9)
    polish_axes(ax, grid_axis="y")
    plt.title(f"{metric} Comparison", pad=10)
    plt.ylim(0, 1)

    for i, v in enumerate(results_df[metric]):
        ax.text(i, v + 0.012, f"{v:.2f}", ha="center", va="bottom", fontsize=8, color="#2B2B2B")

    plt.xticks(rotation=12)
    plt.tight_layout()
    plt.savefig("Accuracy_Comparison.png", dpi=300, bbox_inches='tight')
    plt.show()
```

```
# 11. ROC CURVE
plt.figure(figsize=(6.6, 4.2))
ax = plt.gca()

for (name, y_prob), color in zip(pred_store.items(), LINE_COLORS):
    fpr, tpr, _ = roc_curve(y_test, y_prob)
    auc_val = roc_auc_score(y_test, y_prob)
    plt.plot(fpr, tpr, label=f"{name} (AUC = {auc_val:.3f})", color=color, linewidth=2)

plt.plot([0, 1], [0, 1], "--", color="#B0B0B0", linewidth=1.2)
plt.title("ROC Curve Comparison", pad=10)
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
polish_axes(ax, grid_axis="both")
plt.legend(loc="lower right")
plt.tight_layout()
plt.savefig("ROC_Curve_Comparison.png", dpi=300, bbox_inches='tight')
plt.show()

# 12. PRECISION-RECALL CURVE
plt.figure(figsize=(6.6, 4.2))
ax = plt.gca()

for (name, y_prob), color in zip(pred_store.items(), LINE_COLORS):
    precision, recall, _ = precision_recall_curve(y_test, y_prob)
    pr_auc = average_precision_score(y_test, y_prob)
```

```
    plt.plot(recall, precision, label=f"{name} (AP = {pr_auc:.3f})", color=color, linewidth=2)

baseline = y_test.mean()
plt.axhline(baseline, linestyle="--", color="#B0B0B0", linewidth=1.2, label=f"Baseline =
{baseline:.3f}")
plt.title("Precision-Recall Curve Comparison", pad=10)
plt.xlabel("Recall")
plt.ylabel("Precision")
polish_axes(ax, grid_axis="both")
plt.legend(loc="lower left")
plt.tight_layout()
plt.savefig("Precision-Recall_Curve_Comparison.png", dpi=300, bbox_inches='tight')
plt.show()

# 13. CALIBRATION PLOT
plt.figure(figsize=(6.6, 4.2))
ax = plt.gca()

for (name, y_prob), color in zip(pred_store.items(), LINE_COLORS):
    prob_true, prob_pred = calibration_curve(y_test, y_prob, n_bins=10, strategy="uniform")
    plt.plot(prob_pred, prob_true, marker="o", markersize=5, label=name, color=color,
linewidth=2)

plt.plot([0, 1], [0, 1], "--", color="#B0B0B0", linewidth=1.2)
plt.title("Calibration Plot", pad=10)
plt.xlabel("Mean Predicted Probability")
plt.ylabel("Observed Delay Rate")
polish_axes(ax, grid_axis="both")
```

```
plt.legend(loc="upper left")
plt.tight_layout()
plt.savefig("Calibration_Plot.png", dpi=300, bbox_inches='tight')
plt.show()

print("\nPROBABILITY-FORECASTING METRICS")
print(results_df[["Model", "BrierScore", "LogLoss"]].sort_values("BrierScore").round(4))

# 14. BEST MODEL SELECTION
best_model_name = results_df.sort_values("ROC_AUC", ascending=False).iloc[0]["Model"]
best_probs = pred_store[best_model_name]

print(f"\nBest model by ROC-AUC: {best_model_name}")

best_calibrated_name = results_df.sort_values("BrierScore", ascending=True).iloc[0]["Model"]
print(f"Best model by Brier Score: {best_calibrated_name}")

# 15. THRESHOLD ANALYSIS FOR THE BEST DISCRIMINATION MODEL
thresholds = [0.3, 0.5, 0.7]
threshold_rows = []

for t in thresholds:
    y_pred_t = (best_probs >= t).astype(int)
    tn, fp, fn, tp = confusion_matrix(y_test, y_pred_t).ravel()

    threshold_rows.append({
        "Threshold": t,
```

```
        "Accuracy": accuracy_score(y_test, y_pred_t),
        "Precision": precision_score(y_test, y_pred_t, zero_division=0),
        "Recall": recall_score(y_test, y_pred_t, zero_division=0),
        "F1-score": f1_score(y_test, y_pred_t, zero_division=0),
        "TN": tn,
        "FP": fp,
        "FN": fn,
        "TP": tp
    })

threshold_df = pd.DataFrame(threshold_rows)

print(f"\nTHRESHOLD ANALYSIS FOR: {best_model_name}")
print(threshold_df.round(3))

# 16. CONFUSION MATRICES AT MULTIPLE THRESHOLDS
fig, axes = plt.subplots(1, 3, figsize=(15, 4.3))

for ax, t in zip(axes, thresholds):
    y_pred_t = (best_probs >= t).astype(int)
    cm = confusion_matrix(y_test, y_pred_t)

    cm_percent = cm.astype(float) / cm.sum(axis=1, keepdims=True) * 100

    labels = np.array([
        [f"{cm[0,0]}\n({cm_percent[0,0]:.1f}%)", f"{cm[0,1]}\n({cm_percent[0,1]:.1f}%)"],
        [f"{cm[1,0]}\n({cm_percent[1,0]:.1f}%)", f"{cm[1,1]}\n({cm_percent[1,1]:.1f}%)"]
```

```
    ])

    sns.heatmap(
        cm,
        annot=labels,
        fmt="",
        cmap=CMAP_SOFT,
        cbar=False,
        xticklabels=["Predicted: No Delay", "Predicted: Delay"],
        yticklabels=["Actual: No Delay", "Actual: Delay"],
        linewidths=0.6,
        linecolor="white",
        square=True,
        ax=ax
    )

    ax.set_title(f"{best_model_name}\nThreshold = {t}", pad=10)
    ax.set_xlabel("")
    ax.set_ylabel("")
    ax.tick_params(axis="x", rotation=0, labelsize=9)
    ax.tick_params(axis="y", rotation=0, labelsize=9)
    for spine in ax.spines.values():
        spine.set_visible(False)

plt.tight_layout()
plt.savefig("CONFUSION_MATRICES.png", dpi=300, bbox_inches='tight')
plt.show()
```

```
# 17. FEATURE IMPORTANCE
raw_imp = pd.Series(best_rf.feature_importances_, index=X.columns)

grouped = {
    "carrier": raw_imp[raw_imp.index.str.startswith("carrier_")].sum(),
    "DepHour": raw_imp.get("DepHour", 0),
    "distance": raw_imp.get("distance", 0)
}

if any(col.startswith("origin_") for col in X.columns):
    grouped["origin"] = raw_imp[raw_imp.index.str.startswith("origin_")].sum()

if any(col.startswith("dest_") for col in X.columns):
    grouped["dest"] = raw_imp[raw_imp.index.str.startswith("dest_")].sum()

grouped_imp = pd.Series(grouped).sort_values()

plt.figure(figsize=(6.2, 4.0))
ax = sns.barplot(x=grouped_imp.values, y=grouped_imp.index, color=BAR_COLOR,
saturation=0.9)
polish_axes(ax, grid_axis="x")
plt.title("Grouped Feature Importance", pad=10)
plt.xlabel("Importance")
plt.ylabel("")

for i, v in enumerate(grouped_imp.values):
```

```
    ax.text(v + 0.005, i, f"{v:.2f}", va="center", fontsize=8, color="#2B2B2B")

plt.tight_layout()
plt.savefig("Grouped_Feature_Importance.png", dpi=300, bbox_inches='tight')
plt.show()

print("\nGrouped Feature Importance")
print(grouped_imp.round(3))

# 18. DELAY PATTERN ANALYSIS
airline_delay = df.groupby("carrier")["Delay"].mean().sort_values(ascending=False)

plt.figure(figsize=(6.6, 4.1))
ax = sns.barplot(x=airline_delay.index, y=airline_delay.values, color=BAR_COLOR,
saturation=0.9)
polish_axes(ax, grid_axis="y")
plt.title("Delay Probability by Airline", pad=10)
plt.xlabel("Airline")
plt.ylabel("Probability of Delay")

for i, v in enumerate(airline_delay.values):
    ax.text(i, v + 0.008, f"{v:.2f}", ha="center", fontsize=7, color="#2B2B2B")

plt.tight_layout()
plt.savefig("Delay_Probability_by_Airline.png", dpi=300, bbox_inches='tight')
plt.show()
```

```
hour_delay = df.groupby("DepHour")["Delay"].mean()

plt.figure(figsize=(6.6, 4.1))
ax = plt.gca()
plt.plot(hour_delay.index, hour_delay.values, marker="o", color="#1f77b4", linewidth=2,
markersize=5)
plt.title("Delay Probability by Departure Hour", pad=10)
plt.xlabel("Departure Hour")
plt.ylabel("Probability of Delay")
polish_axes(ax, grid_axis="both")

for x, yv in zip(hour_delay.index, hour_delay.values):
    plt.text(x, yv + 0.008, f"{yv:.2f}", ha="center", fontsize=6, color="#2B2B2B")

plt.tight_layout()
plt.savefig("Delay_Probability_by_Departure_Hour.png", dpi=300, bbox_inches='tight')
plt.show()

df["DistanceGroup"] = pd.cut(
    df["distance"],
    bins=[0, 500, 1000, 1500, 2000, df["distance"].max()],
    labels=["0-500", "501-1000", "1001-1500", "1501-2000", "2000+"],
    include_lowest=True
)

distance_delay = df.groupby("DistanceGroup", observed=False)["Delay"].mean()
```

```
plt.figure(figsize=(6.2, 4.0))
ax = sns.barplot(x=distance_delay.index, y=distance_delay.values, color=BAR_COLOR,
saturation=0.9)
polish_axes(ax, grid_axis="y")
plt.title("Delay Probability by Distance Group", pad=10)
plt.xlabel("Distance Group")
plt.ylabel("Probability of Delay")

for i, v in enumerate(distance_delay.values):
    ax.text(i, v + 0.008, f"{v:.2f}", ha="center", fontsize=7, color="#2B2B2B")

plt.tight_layout()
plt.savefig("Delay_Probability_by_Distance_Group.png", dpi=300, bbox_inches='tight')
plt.show()

if "origin" in df.columns:
    airport_delay = df.groupby("origin")["Delay"].mean().sort_values(ascending=False)

    plt.figure(figsize=(5.8, 4.0))
    ax = sns.barplot(x=airport_delay.index, y=airport_delay.values, color=BAR_COLOR,
saturation=0.9)
    polish_axes(ax, grid_axis="y")
    plt.title("Delay Probability by Origin Airport", pad=10)
    plt.xlabel("Origin Airport")
    plt.ylabel("Probability of Delay")

    for i, v in enumerate(airport_delay.values):
        ax.text(i, v + 0.008, f"{v:.2f}", ha="center", fontsize=7, color="#2B2B2B")
```

```
    plt.tight_layout()
    plt.savefig("Delay_Probability_by_Origin_Airport.png", dpi=300, bbox_inches='tight')
    plt.show()

# 19. SUMMARY TABLES
print("\nDelay Probability by Airline")
print(airline_delay.round(3))

print("\nDelay Probability by Departure Hour")
print(hour_delay.round(3))

print("\nDelay Probability by Distance Group")
print(distance_delay.round(3))

if "origin" in df.columns:
    print("\nDelay Probability by Origin Airport")
    print(airport_delay.round(3))

# 20. FINAL INTERPRETATION
print("\nFINAL MODEL INTERPRETATION")
print(f"- Strongest ranking model (ROC-AUC): {best_model_name}")
print(f"- Best probability calibration (lowest Brier score): {best_calibrated_name}")

if best_model_name == best_calibrated_name:
    print(f"- {best_model_name} performs best on both discrimination and probability
forecasting.")
```

```
else:
    print(f"- {best_model_name} ranks delayed flights most effectively,")
    print(f"  while {best_calibrated_name} provides the most reliable probabilities.")

print("\nConclusion:")
print("Confusion matrices at threshold 0.5 remain useful for interpretation,")
print("but they do not fully capture the quality of probabilistic predictions.")
print("Therefore, ROC-AUC, PR-AUC, calibration, Brier score, and log loss")
print("provide a more complete evaluation of flight delay prediction performance.")
```